

DevGen — full presentation script & visual blueprint

Every sub-section of the DevGen B.Sc. CSIT report gets its own dedicated slide, one-to-one. Chapters 2 (Background & Literature Review) and 4 (System Design) are expanded algorithm-by-algorithm and model-by-model, matched with a detailed visual directive for every slide.

62	16	13	6
TOTAL SLIDES	CHAPTER 2 SLIDES	CHAPTER 4 SLIDES	REPORT CHAPTERS

SLIDE 1

Title Slide — DevGen Project Overview

SLIDE TEXT CONTENT

- DevGen: A Hybrid Neuro-Generative Framework for Devanagari Handwritten Text Recognition and Synthesized Word Augmentation — a B.Sc. CSIT project pairing OCR (reading handwriting) with generative AI (manufacturing extra training handwriting).
- Two problems solved together: reading messy handwritten Devanagari accurately, and generating synthetic handwriting because real labeled data is scarce.
- Built and trained entirely on a consumer Apple Silicon laptop — no cloud GPU, no paid compute — proving transformer-scale OCR is now reachable by students and small teams.

VISUAL & ARCHITECTURE BLUEPRINT

Title Graphic / Hero Banner

Full-width banner. Left half: a real handwritten Devanagari word (e.g. नमस्ते) in ink texture. Right half: the same word regenerated by the GAN, subtly stylized to read as 'AI-generated'. A thin arrow between them labeled 'Recognize ⇌ Generate'. Bottom strip: small icons for CNN, Transformer, GAN, Diffusion.

Abstract — The Big Picture

SLIDE TEXT CONTENT

- Devanagari is hard to read by machine: a shared headline (Shirorekha) joins letters, vowel marks (matras) attach in four positions, consonants fuse into conjuncts, and every writer differs.
- DevGen's Smart ImageRouter looks at a crop and instantly decides: single character → small CNN; full word → TrOCR (Transformer OCR) fine-tuned with LoRA. Recognized text then flows into a rule-based NER engine pulling dates, IDs, districts from Nepali documents.
- Two generative models were built to fight data scarcity: an attention-boosted StyleGAN and a Latent Diffusion Model with ControlNet — the GAN won.
- Headline numbers: TrOCR+LoRA reached CER 0.1380 training only 2.45% of parameters (4.62M of 188.7M); CNN hit 98.2% on clean characters but collapsed to 15.71% on characters segmented from real words.

VISUAL & ARCHITECTURE BLUEPRINT

Infographic Summary Panel

Horizontal pipeline: [Image Upload] → [Smart Router (diamond)] splitting into 'Character → CNN → 98.2% clean / 15.71% real-word' and 'Word → TrOCR+LoRA → CER 0.1380'. Side panel 'Data Scarcity Fix' with StyleGAN-Attn (✅ stable) vs LDM-ControlNet (⚠️ unstable), trophy on the GAN.

SLIDE 3

1.1 Introduction

SLIDE TEXT CONTENT

- Printed Latin OCR is basically solved; handwritten Indic scripts like Devanagari (500M+ speakers: Nepali, Hindi, Sanskrit, Marathi) remain genuinely hard.
- Why: letters are physically joined by a top headline (Shirorekha); matras attach above/below/before/after a consonant; consonants fuse into conjunct shapes unlike their parts; handwriting style varies hugely person to person.
- Old-school OCR segments a word into characters before reading — one bad cut breaks the whole reading. DevGen avoids this.
- DevGen's fix: hybrid recognizer (CNN for characters + segmentation-free TrOCR for words, auto-picked by a router) plus generative models for more training data, plus entity extraction for real documents like citizenship certificates and land records.

VISUAL & ARCHITECTURE BLUEPRINT

Conceptual Illustration

One handwritten Devanagari word with labeled callouts: arrow to the top line = 'Shirorekha (joins letters)'; arrow to a mark above a letter = 'Matra (vowel sign)'; arrow to a fused glyph = 'Conjunct (2 consonants merged)'. Below: three miniature samples of the same word from different writers showing style variation.

1.2 Problem Statement

SLIDE TEXT CONTENT

- Segmentation-based OCR breaks on real handwriting: the Shirerekha is hard to remove cleanly, matras split from or merge into base letters, adjacent strokes bleed together.
- Vision-Transformer OCR sidesteps segmentation by reading the whole image as a sequence — but needs hundreds of millions of parameters, normally trained on GPU clusters most students/small orgs lack.
- Devanagari handwriting datasets are also far rarer than printed or Latin-script data, capping generalization.
- Two real needs tackled here: (a) a recognition pipeline accurate and light enough to train on a laptop, (b) a generative method to manufacture realistic extra training words.

VISUAL & ARCHITECTURE BLUEPRINT

Problem/Cause Flowchart

Left column 'Root Causes': three stacked boxes (Segmentation Fragility, Heavy Compute Requirement, Data Scarcity) arrow into a central box 'Barrier to Devanagari OCR Progress', which branches down into 'Need 1: Efficient Trainable Recognizer' and 'Need 2: Synthetic Data Generator', both feeding a bottom box 'DevGen'.

1.3 Objectives

SLIDE TEXT CONTENT

- Design a hybrid OCR pipeline: rule-based router + lightweight CNN (characters) + LoRA-tuned TrOCR (words).
- Implement parameter-efficient fine-tuning (PEFT) locally on macOS via PyTorch's MPS (Metal) backend — no cloud GPU.
- Build a conditional word-image generator: StyleGAN with attention, guided by a CTC text-recognizer loss so generated strokes actually spell the target word.
- Evaluate an LDM with ControlNet conditioning and rigorously compare it against the GAN.
- Extract structured entities (dates, citizenship IDs, districts, plot numbers) via rule-based NER.
- Deploy everything through a FastAPI backend + React frontend for real-time inference, evaluation, and analysis.

VISUAL & ARCHITECTURE BLUEPRINT

Checklist / Roadmap Graphic

Six numbered milestone icons on a horizontal road/timeline: router-fork, laptop-with-chip, paintbrush+text, diffusion-noise, magnifying-glass-over-document, server+browser — each sitting on the road in build order.

1.4 Scope and Limitation

SLIDE TEXT CONTENT

- In scope: single-character recognition (46 classes: 36 consonants + 10 digits) and word-crop recognition; preprocessing, smart routing, CNN classification, TrOCR decoding, entity extraction, CER evaluation, dataset browsing, two generation pathways — all via a web UI.
- Out of scope: no full-document layout analysis, no text-line detection, no paragraph segmentation — input must already be a pre-cropped character or word. Generated images fixed at 256×64 grayscale.
- Validation CER during training used a 1,000-sample dev subset (not the full test set) for MPS speed — treat as indicative, not final.
- NER is regex-based (only coded patterns caught); on-screen confidence comes from beam-search scores, not a calibrated probability.

VISUAL & ARCHITECTURE BLUEPRINT

Two-Column Comparison Diagram

Left green-boundary column  'In Scope' with icons: character crop, word crop, NER tags, CER meter, dataset browser, generator. Right amber-boundary column  'Limitations' with icons: full page with red X (no layout parsing), fixed 256×64 lock icon, 1,000/12,869 partial-sample gauge.

1.5 Development Methodology

SLIDE TEXT CONTENT

- Incremental, iterative build — modules built and tested one at a time; earlier modules revisited when later ones exposed gaps (e.g. inference preprocessing reworked to match training distribution).
- Phase 1–2: requirement study + dataset acquisition (DHCD, IIIT-INDIC-HW-WORDS); preprocessing + smart routing (adaptive threshold, deskew, denoise, flood-fill router).
- Phase 3–4: recognition models — CNN on DHCD, rank-16 LoRA on Devanagari TrOCR; generative modeling — StyleGAN-Attention stabilized with hinge loss, spectral norm, DiffAugment, CTC guidance.
- Phase 5–7: LDM ControlNet exploration on rendered-glyph control images; integration via FastAPI + React/Vite; testing — CER, classification accuracy, precision/recall/F1, failure-mode analysis.

VISUAL & ARCHITECTURE BLUEPRINT

Iterative Loop Flowchart

Circular/spiral flow of 7 numbered stages (Requirements → Preprocessing → Recognition → Generative → LDM → Integration → Testing) with a curved feedback arrow looping from stage 3 back to stage 2, labeled 'revisit preprocessing after distribution mismatch found'.

1.6 Report Organization

SLIDE TEXT CONTENT

- Ch.1 — introduction, motivation, objectives, scope, methodology.
- Ch.2 — theoretical background + literature review. Ch.3 — requirement, feasibility, and object-oriented analysis with diagrams.
- Ch.4 — refined design + mathematical/algorithmic formulations. Ch.5 — implementation tools, module details, testing, result analysis.
- Ch.6 — conclusion + future recommendations, followed by references, bibliography, and appendices (code excerpts, API reference, paper links, supervisor visit log).

VISUAL & ARCHITECTURE BLUEPRINT

Book/Chapter Map

Vertical stacked book-spine graphic, 6 colored chapter tabs each with a one-icon summary (lightbulb=Ch1, books=Ch2, checklist=Ch3, gears=Ch4, wrench=Ch5, flag=Ch6). Small folder icon at bottom labeled 'References / Appendices'.

SLIDE 9

DEEP DIVE

2.1.1 The Devanagari Script

SLIDE TEXT CONTENT

- Devanagari is an abugida — each symbol is a consonant already carrying a built-in vowel sound, modifiable by attachable vowel marks.
- 36 base consonants (व्यंजन), 12–14 vowels (स्वर) with dependent matras, 10 numerals (०–९).
- Four machine-hard properties: continuous Shirorekha headline; matras attaching in 4 positions; conjunct clusters (संयुक्ताक्षर) fusing 2+ consonants into new shapes; heavy inter-writer variability.
- Net effect: the number of visually distinct glyph shapes vastly exceeds the naive ‘36 letters’ count.

VISUAL & ARCHITECTURE BLUEPRINT

Annotated Script Diagram

Large rendering of a Devanagari word with color overlays: red line tracing the Shirorekha; blue circles around matra positions (above/below/side); green box around a conjunct cluster with a before/after mini-comparison of the two separate consonants vs. the fused form.

2.1.2 Digital Image Preprocessing

SLIDE TEXT CONTENT

- Grayscale conversion collapses color to one intensity channel, simplifying everything downstream.
- Adaptive thresholding binarizes using a locally computed threshold $T(x,y) = \mu_G(x,y) - C$ (μ_G = Gaussian-weighted neighborhood mean) — more robust than one global threshold under uneven lighting.
- Deskewing detects dominant near-horizontal lines via the Hough transform in (ρ, θ) space, then rotates by the negative of the estimated skew angle.
- Non-local means denoising removes speckle by averaging pixels with similar surrounding patches, not just nearby ones.

VISUAL & ARCHITECTURE BLUEPRINT

Before/After Processing Pipeline

Four-panel filmstrip of the same handwriting sample: Raw color scan → Grayscale → Binarized (adaptive threshold, with inset showing the Gaussian window sliding over the image) → Deskewed + Denoised (faint rotated grid line showing the correction angle).

2.1.3 Convolutional Neural Networks

SLIDE TEXT CONTENT

- A CNN learns small filters (kernels) sliding over an image to detect edges/strokes: $S(i,j) = \sum \sum X(i+m,j+n) \cdot K(m,n) + b$.
- Standard stack: Convolution $\rightarrow$ ReLU ($\max(0,x)$) $\rightarrow$ Max Pooling (shrinks spatial size) $\rightarrow$ Batch Normalization $\rightarrow$ Dropout (prevents overfitting).
- Final layer: Softmax $\sigma(z)_i = e^{z_i} / \sum e^{z_j}$ turns raw scores into probabilities, trained by minimizing cross-entropy loss $L = -\sum y_i \log \hat{y}_i$.
- DevGen's character classifier follows exactly this pattern, compact in 3 blocks.

VISUAL & ARCHITECTURE BLUEPRINT

Layer-by-Layer Architecture Diagram

Horizontal block diagram: input tile $\rightarrow$ sliding-kernel animation icon over a small grid (convolution) $\rightarrow$ ReLU kink-graph icon $\rightarrow$ pooling icon (2x2 grid collapsing to 1x1) $\rightarrow$ repeated as 'Block' $\rightarrow$ final Softmax bar-chart showing probability spikes across 46 classes.

2.1.4 Recurrent Networks and the GRU

SLIDE TEXT CONTENT

- RNNs process a sequence step by step, carrying a hidden state forward — plain RNNs suffer vanishing gradients over long sequences.
- GRU fixes this with two gates: update gate $z_t = \sigma(W_z x_t + U_z h_{t-1})$ decides how much old memory to keep; reset gate $r_t = \sigma(W_r x_t + U_r h_{t-1})$ decides how much to forget for the candidate state.
- Candidate + final state: $\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}))$; $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$.
- Bidirectional GRU runs one GRU forward, one backward, and concatenates — every position sees both left and right context. DevGen uses this as the generator's spatial text encoder.

VISUAL & ARCHITECTURE BLUEPRINT

Gate Flow Diagram

A single GRU cell box with two gate valves 'Update Gate z_t ' and 'Reset Gate r_t ', arrows for x_t in, h_{t-1} in, h_t out. Below: a chain of cells reading left-to-right (blue, forward) and a mirrored chain reading right-to-left (orange, backward), merging at each timestep into a concatenated bidirectional output.

2.1.5 Attention and the Transformer

SLIDE TEXT CONTENT

- Transformers replace recurrence with self-attention: every position can directly look at every other position in one step.
- Core formula: $\text{Attention}(Q, K, V) = \text{Softmax}(QK^T / \sqrt{d_k}) \cdot V$ — queries/keys score relevance, softmax weighs, weights combine values.
- Multi-head attention runs several such computations in parallel to capture different relationships at once; positional encodings inject order info attention itself lacks.
- Why it matters here: the Shirorekha spans a whole word, so the model needs long-range context — exactly what self-attention captures well.

VISUAL & ARCHITECTURE BLUEPRINT

Attention Matrix Illustration

Grid/heatmap diagram with image patches or character positions on both axes, connecting lines of varying thickness (attention weight strength) between distant positions — a thick line connecting the first and last letter of a word through their shared Shirorekha.

2.1.6 TrOCR

SLIDE TEXT CONTENT

- TrOCR treats OCR as image-to-text sequence generation, not per-character classification.
- ViT encoder: splits the image into fixed-size patches, embeds each as a vector, processes through Transformer encoder layers.
- Autoregressive Transformer decoder generates the output sequence one token at a time, attending back to encoder image features at every step.
- No explicit character segmentation needed — ideal for cursive, Shirorekha-joined Devanagari words. At inference, beam search keeps the b most probable partial sequences at each step.

VISUAL & ARCHITECTURE BLUEPRINT

Encoder-Decoder Architecture Diagram

Left: input word image chopped into a patch grid feeding 'ViT Encoder' stack. Middle arrow 'image features' into right block 'Transformer Decoder' outputting characters one at a time (growing string, newest highlighted). Side panel: 'Beam Search' as 4 parallel candidate branches, top-scoring one highlighted.

2.1.7 Transfer Learning and LoRA

SLIDE TEXT CONTENT

- Transfer learning: start from a model pre-trained on huge data, adapt it to your task — saves data and compute.
- Full fine-tuning updates every weight and needs a full-size copy per task — expensive.
- LoRA freezes W_0 and learns a small low-rank update: $W = W_0 + (\alpha/r) \cdot BA$, $B \in \mathbb{R}^{(d \times r)}$, $A \in \mathbb{R}^{(r \times k)}$, rank $r \ll \min(d, k)$. Only A and B train.
- Result: 1–2 orders of magnitude fewer trainable parameters and a small swappable adapter — exactly what makes local Apple Silicon fine-tuning of a 188M-parameter model feasible.

VISUAL & ARCHITECTURE BLUEPRINT

Matrix Decomposition Diagram

Large frozen weight matrix W_0 (grayed, padlock icon) with a parallel path below: two thin matrices B (tall-skinny) and A (short-wide) multiplying into a low-rank correction ΔW , added (+) back to W_0 . Rank r shown as a visible thin bottleneck. Side pie chart: 2.45% trainable vs 97.55% frozen.

2.1.8 Generative Adversarial Networks

SLIDE TEXT CONTENT

- GAN = minimax game: $\min_G \max_D \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$ — Generator fools Discriminator, Discriminator catches fakes.
- Stabilizers used: spectral normalization (caps discriminator's Lipschitz constant), hinge loss (margin-based), DiffAugment (differentiable augmentation to stop discriminator overfitting small data), TTUR (discriminator learns faster than generator), EMA (smoother generator weights at inference).
- StyleGAN2 contributes weight demodulation (style vector modulates conv weights, then renormalizes); SAGAN contributes self-attention for long-range structure.
- DevGen's generator combines all of these techniques into one Devanagari-specific architecture.

VISUAL & ARCHITECTURE BLUEPRINT

Adversarial Loop Diagram

Generator G (input noise z) producing a fake image → arrow into Discriminator D (also receiving real images) → D outputs Real/Fake score → feedback loops to both. Badge icons: 🌀 Spectral Norm, 📐 Hinge Loss, 🎨 DiffAugment, ⌚ TTUR, 📊 EMA.

2.1.9 Diffusion Models, LDM, and ControlNet

SLIDE TEXT CONTENT

- Diffusion models reverse a gradual noising process. Forward: $q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{(1-\beta_t)} \cdot x_{t-1}, \beta_t \cdot I)$; a network learns to predict and remove that noise in reverse.
- Latent Diffusion Models do this in a compressed VAE latent space — much cheaper computationally than raw pixels.
- Stable Diffusion conditions generation on a text prompt encoded by CLIP's text encoder.
- ControlNet adds a trainable copy of U-Net encoder blocks, linked via zero-initialized convolutions, to inject an external spatial condition (e.g. a rendered glyph edge map) without destabilizing the base model.

VISUAL & ARCHITECTURE BLUEPRINT

Forward/Reverse Process Diagram + ControlNet Branch

Top row: clean image progressively turning to noise across 5 frames (forward, increasing t), reversed arrow below showing denoising back to clean (reverse). Below: frozen U-Net (gray) with a parallel trainable ControlNet branch (colored) connected by zero-conv plug icons, receiving a rendered Devanagari glyph control image.

2.1.10 Evaluation Metrics

SLIDE TEXT CONTENT

- CER — Levenshtein edit distance normalized by reference length: $CER = (S+D+I)/N$ (Substitutions+Deletions+Insertions over N reference characters). Lower is better.
- Word Exact-Match Accuracy — fraction of words with zero character errors at all.
- Character Precision/Recall/F1: $P = |T_{pred} \cap T_{gt}| / |T_{pred}|$, $R = |T_{pred} \cap T_{gt}| / |T_{gt}|$, $F1 = 2PR / (P+R)$.
- FID for generative quality: $d^2 = ||\mu_r - \mu_g||^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$ — distance between real/generated feature distributions; lower = more realistic.

VISUAL & ARCHITECTURE BLUEPRINT

Metrics Dashboard Mockup

2x2 grid of metric cards: text diff with highlighted S/D/I characters (CER); checkmark/cross tally (word exact-match); small Venn diagram (precision/recall overlap); two overlapping bell curves real vs. generated with a gap labeled 'FID distance'.

2.2 Literature Review — GAN-Based Handwriting Synthesis

SLIDE TEXT CONTENT

- Chhatkuli et al. (2021) first generated Nepali handwritten letters/words with a vanilla GAN, proving feasibility — but limited stroke fidelity, no guarantee the image spells the target text.
- DevGen extends this with spectral normalization, DiffAugment, StyleGAN2 weight demodulation, self-attention, and critically a CTC recognizer-in-the-loop loss to push generated strokes toward actually spelling the conditioning text.
- Relation: this is the direct academic ancestor of DevGen's generative half — a stability, fidelity, and text-correctness upgrade over the baseline.

VISUAL & ARCHITECTURE BLUEPRINT

Before/After Lineage Diagram

Left node 'Chhatkuli et al. 2021 — Vanilla GAN' with a low-fidelity sample glyph. Arrow labeled '+Spectral Norm, +DiffAugment, +StyleGAN2 Demod, +Self-Attention, +CTC Guidance' pointing right to 'DevGen StyleGAN-Attn' with a crisper sample glyph.

2.2 Literature Review — Transformer OCR

SLIDE TEXT CONTENT

- Li et al. (2021) introduced TrOCR, an end-to-end Transformer OCR beating prior CNN-RNN pipelines by dropping explicit segmentation entirely.
- DevGen doesn't use a generic English TrOCR — it adapts a Devanagari-specialized checkpoint, since tokenizer, decoder language prior, and visual distribution differ substantially across scripts.
- This directly explains why LoRA fine-tuning starts from paudelani/trocr-devanagari-2 rather than a stock English checkpoint.

VISUAL & ARCHITECTURE BLUEPRINT

Model Lineage Comparison

Two side-by-side model cards: 'Generic English TrOCR' (Latin tokenizer icon) vs 'Devanagari TrOCR checkpoint' (Devanagari tokenizer icon), highlighted checkmark on the Devanagari one with caption 'chosen by DevGen'.

2.2 Literature Review — Parameter-Efficient Fine-Tuning

SLIDE TEXT CONTENT

- Hu et al. (2021) showed LoRA matches full fine-tuning quality on large generation tasks while training a tiny parameter fraction, producing small portable adapters.
- This directly enables DevGen's entire local-training strategy — without it, fine-tuning a 188.7M-parameter model on a laptop's unified memory wouldn't be practical.
- Relation: core PEFT technique used for the local Apple Silicon workflow ($r=16$, $\alpha=32$).

VISUAL & ARCHITECTURE BLUEPRINT

Feasibility Comparison Bar Chart

Two-bar chart: 'Full Fine-Tuning' bar at ~188.7M trainable params (tall, red, 'needs GPU cluster') vs 'LoRA Fine-Tuning' bar at ~4.6M trainable params (short, green, 'fits on laptop MPS').

2.2 Literature Review — Style-Based & Self-Attention GANs

SLIDE TEXT CONTENT

- Karras et al. (StyleGAN2, 2020) contributed weight demodulation — a cleaner way to inject style into generator convolutions without earlier normalization artifacts.
- Zhang et al. (SAGAN, 2019) contributed self-attention for GANs, letting the generator model long-range spatial structure.
- Supporting tools also adopted: Miyato et al. (spectral norm, 2018), Zhao et al. (DiffAugment, 2020), Heusel et al. (TTUR/FID, 2017).
- DevGen's generator is a synthesis of all these techniques into one Devanagari-specific architecture.

VISUAL & ARCHITECTURE BLUEPRINT

Component Attribution Diagram

Central 'DevGen Generator' box with 5 labeled arrows from citation cards: StyleGAN2 (weight demod), SAGAN (self-attention), Miyato (spectral norm), DiffAugment (augmentation), TTUR/FID (training/eval) — showing DevGen as a convergence point of prior research.

2.2 Literature Review — Diffusion & Controllable Generation

SLIDE TEXT CONTENT

- Rombach et al. (LDM, 2022) and Zhang et al. (ControlNet, 2023) together provide the diffusion pipeline DevGen evaluates as the GAN's alternative.
- Radford et al. (CLIP, 2021) provides Stable Diffusion's text encoder — this project identifies CLIP's total lack of Devanagari representation as a key underperformance cause.
- Relation: glyph-conditioned diffusion was tested and found unstable for handwriting strokes specifically (detailed in Chapter 5).

VISUAL & ARCHITECTURE BLUEPRINT

Pipeline Attribution + Weak-Link Callout

Pipeline: [Text Prompt] → [CLIP Text Encoder] → [LDM/ControlNet] → [Generated Image]. Red warning triangle over 'CLIP Text Encoder' annotated 'No Devanagari training data — weak link identified.'

2.2 Table 2.1 — Comparison of Related Work

SLIDE TEXT CONTENT

- Chhatkuli et al. (2021) — Nepali handwriting synthesis, vanilla GAN → baseline; DevGen adds attention + CTC guidance.
- Li et al. (2021) — General OCR, TrOCR (ViT+Transformer) → DevGen fine-tunes a Devanagari TrOCR with LoRA.
- Hu et al. (2021) — LLM adaptation, LoRA → core PEFT technique used for local training.
- Karras et al. (2020) StyleGAN2 → weight-demod design adopted. Rombach et al. (2022) LDM → evaluated as alternative generator. Zhang et al. (2023) ControlNet → glyph-conditioned diffusion, found unstable for strokes.

VISUAL & ARCHITECTURE BLUEPRINT

Data Table Slide

Clean 4-column table (Work | Task | Method | Relation to DevGen) reproduced exactly as in the report, alternating row shading, 'Relation to DevGen' column highlighted in accent color.

SLIDE 25

3.1 System Analysis — Overview

SLIDE TEXT CONTENT

- System analysis answers two questions: what must the system do, and can it actually be built given time/hardware/budget constraints.
- DevGen was analyzed using the object-oriented approach — described in terms of real classes/objects and their interactions.
- Three parts follow: Requirement Analysis (functional/non-functional), Feasibility Analysis (technical/operational/economic/schedule), Object-Oriented Analysis (class/sequence/activity/state diagrams).

VISUAL & ARCHITECTURE BLUEPRINT

Section Overview Map

Simple 3-node horizontal map: 'Requirements (what)' → 'Feasibility (can we)' → 'OO Analysis (how it behaves)', each a distinct icon connected by arrows.

3.1.1(i) Functional Requirements

SLIDE TEXT CONTENT

- FR1 Image upload; FR2 Preprocessing (grayscale, adaptive threshold, deskew, denoise, foreground crop); FR3 Smart routing (aspect ratio + connected-component analysis).
- FR4 Character classification (46 classes, confidence score); FR5 Word recognition (Devanagari text, per-token confidence); FR6 Entity extraction (dates, IDs, districts, provinces, wards).
- FR7 CER evaluation; FR8 Synthetic generation (StyleGAN + LDM ControlNet); FR9 Dataset browsing (metadata, paginated/random samples).

VISUAL & ARCHITECTURE BLUEPRINT

Use Case Diagram (Figure 3.1) + Requirement List

Stick-figure 'End User' actor connected to 5 oval use-case bubbles: Run Smart OCR, Extract Entities, Evaluate CER, Generate Word, Browse Dataset — enclosed in a 'DevGen' system boundary box. Alongside, a numbered FR1–FR9 checklist.

3.1.1(ii) Non-Functional Requirements

SLIDE TEXT CONTENT

- Performance: routing <1ms; character prediction ~5ms; word recognition ~150ms on Apple Silicon. Model size: LoRA adapter <20MB.
- Accuracy: target CER below 0.15, character-classification accuracy above 97% on clean data.
- Portability: no dedicated CUDA GPU, no cloud dependency at inference. Usability: single-page React dashboard for non-technical users.
- Maintainability: modular backend, one responsibility per file, lazy model loading. Reliability: graceful error handling with meaningful HTTP status codes.

VISUAL & ARCHITECTURE BLUEPRINT

Quality-Attribute Radar/Gauge Panel

Row of 7 gauge/speedometer icons, one per NFR category (Performance, Size, Accuracy, Portability, Usability, Maintainability, Reliability), each needle pointing at its target threshold (e.g. Performance gauge marked '<150ms').

3.1.2 Feasibility Analysis

SLIDE TEXT CONTENT

- Technical: MPS backend gives GPU-accelerated training/inference on Apple Silicon; every model/dataset is open-source; LoRA fits trainable parameters in unified memory. Confirmed by a working prototype.
- Operational: the React dashboard bundles upload, recognition, NER, evaluation, dataset inspection, and generation into one interface requiring zero ML knowledge.
- Economic: all software free/open-source, computation on existing hardware — recurring cost near zero (~NPR 5,000 one-time, mostly electricity/internet).
- Schedule: incremental methodology splits work into independently deliverable modules, fitting one academic semester.

VISUAL & ARCHITECTURE BLUEPRINT

Four-Quadrant Feasibility Wheel

Circular wheel in 4 quadrants (Technical, Operational, Economic, Schedule), each shaded green '✓ Feasible' with a justification icon: chip (MPS), dashboard (single UI), coin (~NPR 5,000), calendar (16 weeks).

3.1.2 Feasibility Data — Requirements & Cost

SLIDE TEXT CONTENT

- Hardware: Apple Silicon or x86-64 ≥ 8 cores; ≥ 16 GB RAM; ≥ 10 GB storage. Software: macOS 13+ / Linux; Python 3.10+, Node.js 18+; PyTorch 2.0+, Transformers, PEFT, Diffusers, FastAPI, React, Vite.
- Cost: dev hardware NPR 0 (owned); software NPR 0 (open-source); cloud compute NPR 0 (local MPS); electricity+internet $\approx$ NPR 5,000. Total $\approx$ NPR 5,000.

VISUAL & ARCHITECTURE BLUEPRINT

Two-Table Data Slide

Left table reproduces Table 3.2 (hardware/software, 6 rows). Right table reproduces Table 3.3 (cost breakdown, 4 line items + bolded total), styled like a simple receipt with the total row highlighted.

3.1.2 Feasibility Data — Project Schedule

SLIDE TEXT CONTENT

- Weeks 1–2: Requirement study + data acquisition → datasets, requirement spec.
- Weeks 3–4: Preprocessing + router → working router. Weeks 5–8: CNN + TrOCR training → trained recognizers.
- Weeks 9–12: Generative models → GAN + LDM experiments. Weeks 13–14: Integration + UI → full-stack app. Weeks 15–16: Testing + report → final report.

VISUAL & ARCHITECTURE BLUEPRINT

Gantt Chart

Horizontal Gantt chart, 6 colored bars spanning their week ranges across a 16-week axis, each labeled with phase name and deliverable tag.

3.1.3 Object-Oriented Analysis — Class Diagram

SLIDE TEXT CONTENT

- ImageRouter decides the recognition path; CharacterClassifier wraps DevanagariCNN; TrOCREngine wraps the vision-encoder-decoder model + processor.
- NERExtractor post-processes recognized text; Preprocessor cleans raw inputs; GANGenerator / LDMEngine handle synthesis.
- This decomposition maps directly onto the backend's actual module files (seen later in Chapter 5).

VISUAL & ARCHITECTURE BLUEPRINT

UML Class Diagram (Figure 3.2)

Six class boxes (ImageRouter, CharacterClassifier, DevanagariCNN, TrOCREngine, NERExtractor, Preprocessor, GANGenerator, LDMEngine) each with key attributes/methods, connected by association/dependency lines.

3.1.3 Object-Oriented Analysis — Sequence Diagram

SLIDE TEXT CONTENT

- Run Smart OCR flow: End User uploads via React dashboard → frontend calls FastAPI backend → backend preprocesses (foreground crop) → TrOCREngine invokes ImageRouter (aspect ratio + blob count).
- Word inputs decoded by TrOCR then passed to NERExtractor; character inputs classified by the CNN.
- Backend returns recognized text, per-token confidence, preprocessed-image preview, extracted entities — frontend renders it all.

VISUAL & ARCHITECTURE BLUEPRINT

UML Sequence Diagram (Figure 3.3)

Vertical lifelines for End User, React Frontend, FastAPI Backend, ImageRouter, TrOCREngine/CNN, NERExtractor, with time-ordered horizontal arrows: upload → preprocess → route → recognize → extract-entities → render-result.

3.1.3 Object-Oriented Analysis — Activity & State Diagrams

SLIDE TEXT CONTENT

- Activity Diagram: step-by-step decision flow from upload through preprocessing, routing, model inference, and result return — including the character-vs-word branch point.
- State Diagram: models a request's lifecycle — Received → Preprocessing → Routed → Recognizing → Extracting Entities → Completed/Error.
- Together they capture both the logical flow of processing and the lifecycle of a request object.

VISUAL & ARCHITECTURE BLUEPRINT

UML Activity (Fig 3.4) + State Diagram (Fig 3.5)

Left: tall vertical activity/swimlane diagram with diamond decision nodes ('Character or Word?') branching into parallel paths that rejoin before 'Return Result'. Right: rounded-rectangle state machine with labeled transitions between states, including an Error state reachable from any stage.

SLIDE 34

DEEP DIVE

4.1 Design — Component & Deployment Architecture

SLIDE TEXT CONTENT

- Backend decomposed into single-responsibility modules coordinated by main.py: preprocessing.py, image_router.py, cnn_model.py, trocr_engine.py, ner_extractor.py, ldm_engine.py, dataset_loader.py.
- Frontend: React + Vite SPA with dedicated pages for recognition, NER, evaluation, dataset exploration, generative suite — talks to the backend purely over JSON REST.
- Model artifacts (TrOCR LoRA adapter, CNN weights, GAN checkpoint, ControlNet checkpoint) live on the filesystem, discovered at runtime; heavy ML models lazily loaded on first use.
- Deployment target: a single Apple Silicon developer machine — no distributed infrastructure needed.

VISUAL & ARCHITECTURE BLUEPRINT

Component (Fig 4.1) + Deployment (Fig 4.2) + Block Architecture (Fig 4.3)

Top: component diagram, labeled boxes per backend module connected to a central main.py orchestrator, separate React frontend box via a 'REST API (JSON)' arrow. Bottom: deployment diagram, one physical node 'Apple Silicon Laptop' containing both processes, with a 'Model Artifacts (filesystem)' disk icon attached.

4.1 Design — Generative Subsystem Architecture

SLIDE TEXT CONTENT

- Generative subsystem designed as two independent, swappable modules — StyleGAN-Attention and LDM ControlNet — droppable in/out without touching the recognition pipeline.
- StyleGAN-Attention Generator/Discriminator (Fig 4.4): combines spatial text encoding, style mapping, attention, and weight demodulation (detailed next in 4.2.5).
- LDM ControlNet Pipeline (Fig 4.5): Stable Diffusion v1-5 + ControlNet conditioning on rendered Devanagari glyphs (detailed in 4.2.6).

VISUAL & ARCHITECTURE BLUEPRINT

Dual Pipeline Architecture Diagram (Figs 4.4 + 4.5)

Two parallel horizontal pipelines stacked vertically, both taking the same target word text on the left, producing a 256×64 grayscale image on the right. Top (StyleGAN-Attn): Text Encoder → Style Mapping → Generator (Attn+Demod) → Discriminator (dotted, training-only). Bottom (LDM ControlNet): Glyph Renderer → ControlNet Branch → SD U-Net → VAE Decoder.

4.2.1 Algorithm — Adaptive Gaussian Thresholding

SLIDE TEXT CONTENT

- Formula: $T(x,y) = \mu_G(x,y) - C$, computed over a 15×15 local block, with $C = 10$.
- $\mu_G(x,y)$ is the Gaussian-weighted mean of pixel intensities around (x,y) — the threshold adapts to local lighting/contrast instead of one fixed brightness cutoff.
- First concrete preprocessing algorithm, converting a grayscale scan into a clean binary (ink vs. background) image robust to uneven scanning/lighting.

VISUAL & ARCHITECTURE BLUEPRINT

Step-by-Step Algorithm Flowchart

A sliding 15×15 window icon moving across a grayscale image, zoomed inset showing the Gaussian weight bell-curve over the window's pixels computing μ_G , then a subtraction step ($\mu_G - 10$) producing $T(x,y)$, then a final black/white comparison step.

4.2.1 Algorithm — Deskewing via Hough Transform

SLIDE TEXT CONTENT

- Detects near-horizontal lines by transforming the image into (ρ, θ) parameter space — every straight line maps to a point; peaks correspond to strong lines.
- Skew angle = median angle of detected lines with $|\theta| < 45^\circ$, filtering out irrelevant near-vertical detections.
- Correction: rotate the image by affine matrix M using the estimated angle around image center (x_c, y_c) , straightening the page/word.

VISUAL & ARCHITECTURE BLUEPRINT

Hough Space Transformation Diagram

Left: slightly tilted handwritten text line with a dashed skew-angle line. Middle: corresponding Hough (ρ, θ) accumulator space as a small heatmap with a bright peak marking the dominant angle. Right: the same text rotated back to horizontal via matrix M , before/after slider divider.

4.2.1 Algorithm — Denoising & Foreground Crop

SLIDE TEXT CONTENT

- Non-local means denoising compares each pixel's surrounding patch to other similar patches elsewhere in the image, then averages — preserves stroke edges better than simple blurring.
- OCR foreground crop tightly crops around just the ink region so the input matches the exact word-crop framing the model trained on.
- Why it matters: mismatched training-time vs. inference-time framing was found to materially hurt recognition quality (confirmed by System Test 4).

VISUAL & ARCHITECTURE BLUEPRINT

Patch-Matching + Crop Illustration

Left: noisy image with two highlighted small patches connected by a dashed line, labeled 'similar patches averaged together'. Right: denoised image with a tight bounding box snug around ink strokes labeled 'foreground crop', contrasted against a crossed-out loose padded canvas.

4.2.2 Algorithm — Smart ImageRouter (Flood-Fill Routing)

SLIDE TEXT CONTENT

- Step 1: binarize using $\min(0.75 \cdot \bar{a}, 200)$, compute the ink bounding box. Step 2: $\text{AspectRatio} = W_{\text{bbox}} / H_{\text{bbox}}$.
- Step 3: count connected-component blobs via a stack-based iterative flood fill; a blob only counts if $\text{size } S \geq \max(0.001 \cdot N_{\text{pixels}}, 10)$, filtering tiny noise specks.
- Routing: $\text{AspectRatio} > 2.5$, OR ($\text{AspectRatio} > 1.8$ AND $\text{blobs} \geq 3$), OR $\text{blobs} \geq 4 \rightarrow \text{TrOCR (word)}$. $\text{AspectRatio} < 1.3$ AND $\text{blobs} \leq 2 \rightarrow \text{CNN (character)}$. Ambiguous cases scored by both signals.
- Runs in well under 1ms using pure NumPy — no ML model needed for routing itself.

VISUAL & ARCHITECTURE BLUEPRINT

Decision Flowchart (Figure 1.1)

Decision tree: 'Binarize + Compute BBox' $\rightarrow$ 'Compute AspectRatio & Blob Count' $\rightarrow$ diamond 'AspectRatio > 2.5?' (Yes $\rightarrow$ Word, No $\rightarrow$ next) $\rightarrow$ 'AspectRatio > 1.8 AND blobs ≥ 3 ?' $\rightarrow$ 'blobs ≥ 4 ?' $\rightarrow$ 'AspectRatio < 1.3 AND blobs ≤ 2 ?' $\rightarrow$ Ambiguous scoring box. End nodes colored: blue Word $\rightarrow$ TrOCR, green Character $\rightarrow$ CNN.

4.2.3 Algorithm — CNN Character Classifier Architecture

SLIDE TEXT CONTENT

- Input: $1 \times 32 \times 32$ grayscale character image; output: probabilities over $C=46$ classes (36 letters + 10 digits).
- Block 1: $2 \times \text{Conv2D}(32, 3 \times 3) \rightarrow \text{BatchNorm} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2 \times 2) \rightarrow \text{Dropout}(0.25)$. Block 2: $2 \times \text{Conv2D}(64, 3 \times 3) \rightarrow \text{same}$. Block 3: $\text{Conv2D}(128, 3 \times 3) \rightarrow \text{BatchNorm} \rightarrow \text{ReLU} \rightarrow \text{AdaptiveAvgPool}(4 \times 4) \rightarrow \text{Dropout}(0.25)$.
- Head: $\text{Flatten} \rightarrow \text{Linear}(2048, 256) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.5) \rightarrow \text{Linear}(256, 46)$. Trained by minimizing cross-entropy with Adam.

VISUAL & ARCHITECTURE BLUEPRINT

Layered Network Architecture Diagram

Vertical stack of 4 blocks (Block 1, 2, 3, Head), each a set of stacked rectangular layer icons shrinking in width (spatial size) as depth (channels) grows: $32 \times 32 \times 1 \rightarrow 16 \times 16 \times 32 \rightarrow 8 \times 8 \times 64 \rightarrow 4 \times 4 \times 128 \rightarrow \text{flattened}$
 $2048 \rightarrow 256 \rightarrow 46$ output spikes.

4.2.4 Algorithm — TrOCR LoRA Fine-Tuning Math

SLIDE TEXT CONTENT

- Adapted modules: self-attention Query, Key, Value, Output/Dense projections.
- Formula: $W = W_0 + (\alpha/r)(BA)$, frozen W_0 , rank $r=16$, scaling $\alpha=32$. Only A and B train.
- Result: 2.45% trainable-parameter ratio — 4,620,288 trainable of 188,697,376 total.
- Inference: decoder emits tokens autoregressively; beam search width 4 keeps the four most probable partial sequences; per-token confidence from the softmax scores of the selected sequence.

VISUAL & ARCHITECTURE BLUEPRINT

LoRA Injection Diagram on Transformer Block

Transformer self-attention block (Q/K/V/Output weight matrices), each with a parallel LoRA branch ($B \times A$ low-rank pair) feeding a summation junction (+) before the projection output, annotated $r=16$, $\alpha=32$. Side donut chart: 4.62M trainable (colored) vs 184.1M frozen (gray).

4.2.5 Algorithm — StyleGAN-Attn Spatial Text Encoder

SLIDE TEXT CONTENT

- Purpose: convert the target text (character token sequence) into a spatial feature representation the generator can condition on.
- Token indices t are embedded, then passed through a bidirectional GRU (see Slide 12).
- A linear layer produces $\text{spatial_feats} \in \mathbb{R}^{(512 \times 1 \times L)}$ — a per-position feature map spanning the word's length L .
- Concatenated final hidden states from both GRU directions form a single global feature vector summarizing the whole word.

VISUAL & ARCHITECTURE BLUEPRINT

Encoder Pipeline Diagram

Row of character tokens (e.g. न-म-स्-ते) each converted into an embedding vector icon, feeding a bidirectional GRU chain. Output splits: 'spatial_feats (512×1×L)' to the generator's spatial pathway, 'global feature vector' to a separate conditioning pathway.

4.2.5 Algorithm — StyleGAN-Attn Style Mapping Network

SLIDE TEXT CONTENT

- A 6-layer MLP maps random pixel-normalized noise $z \sim \mathcal{N}(0, I)$ into a style vector w .
- Normalization: $z' = z / \sqrt{((1/d) \cdot \sum z_i^2 + 10^{-8})}$ — rescales the noise vector before it enters the MLP, stabilizing training.
- The resulting style vector w modulates the generator's convolution layers (see next slide) — the classic StyleGAN disentangled style-space idea.

VISUAL & ARCHITECTURE BLUEPRINT

MLP Mapping Diagram

Left: random noise vector z icon through a normalization box (formula shown). Right: 6 stacked rectangular layers 'FC Layer 1' through '6' processing sequentially, ending in a highlighted output vector 'style vector w ' with an arrow off-slide toward 'injected into every generator layer'.

4.2.5 Algorithm — StyleGAN-Attn Generator Modules

SLIDE TEXT CONTENT

- Lightweight bottleneck self-attention: $\text{Softmax}(QK^T/\sqrt{d_k}) \cdot V$, channels compressed to $C/8$ first to keep compute cheap while capturing long-range stroke structure.
- StyleGAN2 weight demodulation: $w''_{ijk} = s_i \cdot w_{ijk} / \sqrt{(\sum_k (s_i \cdot w_{ijk})^2 + 10^{-8})}$ — the style vector scales conv weights, then renormalizes, keeping style and content cleanly separated.
- Trainable noise injection adds small per-pixel random variation per layer for fine stroke texture.
- Multi-scale toRGB skip projections: outputs from multiple resolutions projected to image space and summed, reducing vanishing-gradient problems in deep generators.

VISUAL & ARCHITECTURE BLUEPRINT

Generator Block Internal Diagram (part of Fig 4.4)

Single generator block cross-section: incoming feature map → weight-demodulated convolution (style vector s injected as side input) → noise injection (speckle icon) → bottleneck self-attention (Q/K/V mini diagram, 'C/8' label) → output feature map with a 'toRGB' branch tapping into a running sum accumulating contributions across resolution scales.

4.2.5 Algorithm — StyleGAN-Attn Stabilization & Loss

SLIDE TEXT CONTENT

- Hinge loss with CTC guidance. Discriminator: $L_D = \mathbb{E}[\max(0, 1 - D(x, c))] + \mathbb{E}[\max(0, 1 + D(G(z, c), c))]$ — pushes real scores above +1, fake scores below -1.
- Generator: $L_G = -\mathbb{E}[D(G(z, c), c)] + \lambda \cdot L_{CTC}(G(z, c))$ — the CTC term, from a pre-trained TextRecognizer, forces generated strokes to actually match the target text.
- Extra stabilizers: translation-only DiffAugment, TTUR with $\eta_G = 1 \times 10^{-4}$ and $\eta_D = 4 \times 10^{-4}$ (discriminator learns 4× faster), EMA weight tracking for smoother inference outputs.

VISUAL & ARCHITECTURE BLUEPRINT

Loss Function Breakdown Diagram

Two side-by-side formula cards, Discriminator Loss and Generator Loss, color-coded terms matched to a small diagram: real image → D → score (real term); fake image → D → score (fake term); fake image → pretrained TextRecognizer → CTC loss (guidance term) with a dashed arrow labeled 'λ weight' feeding the generator loss.

4.2.6 Algorithm — Latent Diffusion Model with ControlNet

SLIDE TEXT CONTENT

- Base pipeline: Stable Diffusion v1-5.
- Input Devanagari text is rendered onto a black canvas with a Devanagari TrueType font to form a control image — a literal picture of the target glyph shapes.
- The same text is phonetically transliterated to a Latin prompt for CLIP (e.g. नमस्ते → “handwritten Devanagari word 'namaste' on white paper”), since CLIP can't read Devanagari directly.
- A ControlNet zero-convolution branch injects the rendered glyph structure into the U-Net at each encoder resolution, steering denoising toward the target shapes; the VAE decoder maps the final latent back to pixels.

VISUAL & ARCHITECTURE BLUEPRINT

End-to-End Diffusion Pipeline (Figure 4.5)

Input word नमस्ते splitting into two paths: top rendered as a black-canvas glyph image (control image), bottom transliterated to 'namaste' feeding CLIP. Both converge into the ControlNet-augmented U-Net (frozen U-Net + trainable branch via zero-conv plugs). Output through VAE Decoder into the final generated word image.

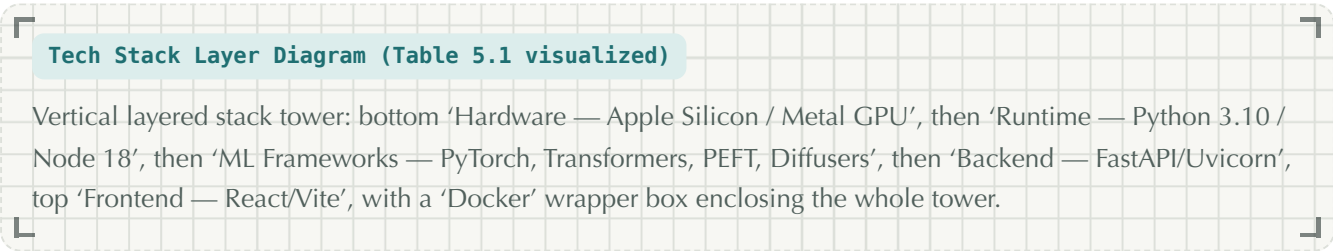
SLIDE 47

5.1.1 Tools Used — Technology Stack

SLIDE TEXT CONTENT

- Backend: Python 3.10+. Frontend: TypeScript/JavaScript (ES2020).
- Deep learning: PyTorch 2.0+ (MPS enabled). Fine-tuning: Hugging Face PEFT (LoRA), $r=16$, $\alpha=32$. OCR model: Transformers (TrOCR / VisionEncoderDecoderModel).
- Diffusion: Diffusers — Stable Diffusion v1-5 + ControlNet. Image processing: OpenCV, Pillow, NumPy.
- Serving/UI: FastAPI + Uvicorn (REST API), React + Vite (SPA). Packaging: Docker + docker-compose. Hardware: Apple Silicon macOS, Metal GPU via MPS.

VISUAL & ARCHITECTURE BLUEPRINT



5.1.1 Dataset Split Sizes

SLIDE TEXT CONTENT

- Dataset: IIIT-INDIC-HW-WORDS-Hindi, extracted locally.
- Train: 69,853 rows. Validation: 12,708 rows. Test: 12,869 rows.
- Total: 95,430 labeled word images — the foundation for both TrOCR fine-tuning and evaluation.

VISUAL & ARCHITECTURE BLUEPRINT

Stacked Bar / Pie Split Chart

Single horizontal stacked bar spanning full dataset width, divided into 3 proportional colored segments (Train ~73%, Validation ~13%, Test ~13%), each labeled with its exact row count, small 'Total: 95,430' label above.

5.1.2 Implementation — Smart Router & Preprocessing Modules

SLIDE TEXT CONTENT

- `image_router.py` — `classify_input_type()` converts to grayscale, binarizes with adaptive threshold, computes bounding box + aspect ratio, counts blobs via iterative flood fill (`_count_blobs`, `_flood_fill`). Returns route + confidence + human-readable reason code. Well under 1ms, pure NumPy.
- `preprocessing.py` — `full_preprocess()` for document-style cleanup; `preprocess_for_ocr()` for tight word-crop foreground cropping.
- These two modules are the 'front door' every image passes through before any neural network sees it.

VISUAL & ARCHITECTURE BLUEPRINT

Module Internals Code-Map Diagram

Two file-icon boxes labeled with filenames, each showing key exported functions as pill labels underneath (`classify_input_type`, `_count_blobs`, `_flood_fill` under `image_router.py`; `full_preprocess`, `preprocess_for_ocr` under `preprocessing.py`), connected by a data-flow arrow chain.

5.1.2 Implementation — CNN & TrOCR Engine Modules

SLIDE TEXT CONTENT

- `cnn_model.py` — defines DevanagariCNN (Table 4.1) plus DHCD-style binarization and contour cropping so inference inputs match the training distribution of centered 32×32 images.
- `trocr_engine.py` — runtime artifact inspection, automatic checkpoint discovery, explicit adapter selection via `TROCR_ADAPTER_PATH` env var, MPS device selection with CPU fallback, LoRA loading (optional merge), beam-search generation, token-level confidence. Also hosts CERCalculator (edit-distance CER) and a manual TrOCRProcessor fallback for version mismatches.
- `train_trocr.py` — seq2seq trainer: HF or local dataset loading, LoRA attachment, CER computation, MPS-compatible training args, best-checkpoint tracking by validation CER. Saves adapters with `save_embedding_layers=False` to dodge a known PEFT/VisionEncoderDecoderConfig bug.

VISUAL & ARCHITECTURE BLUEPRINT

Module Internals Code-Map Diagram

Three file-icon boxes (`cnn_model.py`, `trocr_engine.py`, `train_trocr.py`) each listing key responsibilities as bullet tags. Callout beside `trocr_engine.py`: ‘`TROCR_ADAPTER_PATH` env var → selects which LoRA adapter loads at runtime’, small env-variable icon.

5.1.2 Implementation — NER, Dataset Loader & API

SLIDE TEXT CONTENT

- `ner_extractor.py` — regex extraction of Nepali dates (Devanagari ०–९ and ASCII digits), citizenship numbers, plot (killa) and sheet (paana) numbers, districts, provinces, wards, plus a catch-all for other multi-digit IDs and a summary formatter.
- `dataset_loader.py` — metadata, single-sample, random, and paginated access used by the dataset-explorer UI.
- `main.py` — the FastAPI app exposing health, dataset, preprocess, recognize, recognize/full, evaluate, NER, and generate (GAN/LDM) endpoints, with lazy model loading and permissive CORS for local dev.

VISUAL & ARCHITECTURE BLUEPRINT

API Surface Map

Central `main.py` hub icon with spokes to each endpoint group (Health, Dataset, Preprocess, Recognize, Evaluate, NER, Generate), each spoke ending in a small labeled endpoint pill (reflecting the full Appendix C list).

5.1.2 Implementation — Generative Modules & Training Run Config

SLIDE TEXT CONTENT

- StyleGAN-Attention loader (in main.py) reconstructs AdvancedStyleGANAttentionGenerator, SpatialTextEncoder, StyleMappingNetwork, preferring the EMA generator when available; trained via DevGen_StyleGAN_Attn.ipynb.
- LDM lives in ldm_engine.py and DevGen_LDM_ControlNet.ipynb.
- Main TrOCR training run: base model paudelani/tocr-devanagari-2, 2 epochs, batch size 4, eval batch 4, grad accumulation 2 (effective batch 8), lr 1e-5, eval-limit 1000, eval/save/logging every 1000/1000/50 steps.
- LoRA config: r=16, lora_alpha=32, lora_dropout=0.05, target_modules=[query,value,key,dense] → 4,620,288 trainable of 188,697,376 total (2.45%).

VISUAL & ARCHITECTURE BLUEPRINT

Training Command Terminal Mockup

Stylized terminal window showing the exact training command with syntax highlighting, side annotation panel calling out each flag (--epochs, --batch-size, --grad-accum, --learning-rate) with a one-line explanation arrow.

5.2.1 Unit Testing

SLIDE TEXT CONTENT

- UT1 Image Routing (aspect ratio 1.02, 1 blob) → character, confidence 0.90, reason 'square_single_blob'. Pass.
- UT2 Character Classification (क) → confidence 0.9852 (expected >0.8). Pass. UT3 TrOCR Word Recognition (शरावती) → CER 0.0. Pass.
- UT4 CER Calculator (नेपाल vs नेपाली) → CER $1/5=0.2$. Pass. UT5 NER Extraction (date + काठमाडौं) → both extracted. Pass.
- UT6 Empty/low-ink image → defaults to character, confidence 0.5, reason 'very_low_ink'. Pass.

VISUAL & ARCHITECTURE BLUEPRINT

Unit Test Report Table

6-row test-report table (Test ID | Input | Expected | Observed | Result) with a green checkmark badge in the Result column for every row, styled like a CI/CD test-runner output panel.

5.2.2 System Testing & Test Summary

SLIDE TEXT CONTENT

- ST1 End-to-end: upload word image → POST /api/v1/recognize/full → text, preprocessed image, per-token confidence, entity summary, all rendered. Pass.
- ST2 Routing dispatch: character crop → CNN, word crop → TrOCR. Pass. ST3 Generation endpoint: base64 grayscale 256×64 images — visually valid strokes, semantic legibility limited. Pass.
- ST4 Preprocessing distribution fix: foreground-cropped input beats a padded 384×384 canvas. Pass. ST5 Model info/health: status, adapter path, device reported. Pass.
- Overall: 6 unit tests + 5 system tests, all passing (Table 5.4).

VISUAL & ARCHITECTURE BLUEPRINT

Test Summary Dashboard (Table 5.4)

11-row combined test summary table (ID | Type | Focus | Result), unit and system tests visually grouped into two colored bands, an overall '11/11 PASS' banner styled like a build-status badge.

5.3.1 Result Analysis — TrOCR Checkpoint Progression

SLIDE TEXT CONTENT

- Checkpoint 9000 (epoch 1.03): CER 0.1409. Checkpoint 10000 (epoch 1.15): CER 0.1471 — a temporary regression. Checkpoint 11000: CER 0.1457.
- Checkpoint 12000–13000: recovers gradually (0.1428, 0.1440). Checkpoint 14000 (epoch 1.60): best CER 0.1380, loss 0.2339.
- Lesson: the curve is non-monotonic — exactly why checkpoints must be saved and selected by validation CER, not just taken from the final training step.

VISUAL & ARCHITECTURE BLUEPRINT

Line Chart – CER vs. Checkpoint/Epoch

X-axis: checkpoint steps 9000→14000. Y-axis: Validation CER. Line plot with 6 points showing the dip-then-recovery pattern, lowest point (checkpoint 14000, CER 0.1380) highlighted with a star marker and callout 'Best Checkpoint Selected'.

5.3.2 Result Analysis — Diagnostic Metrics

SLIDE TEXT CONTENT

- TrOCR+LoRA (50-sample test subset): CER 0.3955, Word Exact Match 46.00%, Precision 0.7477, Recall 0.8588, F1 0.7910 — high recall = strong stroke sensitivity, precision below recall = some insertion errors.
- Custom CNN: 98.20% clean DHCD accuracy vs 15.71% on real-word-segmented characters; Macro Precision 0.3155, Recall 0.1463, F1 0.1753 — severe fragility once inputs leave the clean distribution.
- StyleGAN-Attn OCR-read correctness: Synthetic Image CER 2.3226, Exact Match 0.00% — measures whether TrOCR can correctly read back the intended word.
- Three GAN-quality lenses considered: FID, precision/recall for GANs, OCR-based semantic accuracy — the last is judged most decisive for handwriting.

VISUAL & ARCHITECTURE BLUEPRINT

Three-Panel Metrics Comparison

Three cards mirroring Tables 5.5–5.7: ‘TrOCR+LoRA’ with a 5-metric mini bar chart; ‘Custom CNN’ with a stark two-bar contrast (98.20% clean vs 15.71% real-word, green vs red); ‘StyleGAN-Attn Read-Back’ showing a high CER bar and a 0% exact-match indicator.

5.3.3 Analysis — Why Segmented CNN Performance Degrades

SLIDE TEXT CONTENT

- CNN hits 98.2% on clean, pre-centered DHCD characters but collapses to 15.71% on characters segmented from real words.
- Root cause: the heuristic segmenter removes the Shirorekha via connected components, but real handwriting is irregular — characters slice into sub-pieces, matras get left attached or split off wrongly, adjacent strokes bleed together.
- The resulting off-center, fragmented crops are a severe out-of-distribution shift for a classifier trained only on clean isolated glyphs.
- Bigger picture: quantitative proof that segmentation-free sequence models (TrOCR) are the right architectural choice — the empirical justification for DevGen's hybrid-routing design.

VISUAL & ARCHITECTURE BLUEPRINT

Failure Case Illustration

Side-by-side: clean centered DHCD character (98.2% accurate, green check) vs. a real segmented crop — off-center, sliced, stray matra fragment floating disconnected, red X and '15.71% — OOD shift'. Arrow beneath connecting to a concluding box: '→ justifies segmentation-free TrOCR path'.

5.3.3 Analysis — Semantic Gap in GAN Generation

SLIDE TEXT CONTENT

- The GAN produces crisp, realistic strokes and convincing Devanagari shapes — high visual fidelity.
- But scores a high OCR-read CER (2.3226) and 0% exact match — the generated image often doesn't actually spell the conditioned word.
- Why: the mapping from conditioning tokens to physical character layout isn't tightly constrained, so TrOCR's own language-model priors bias it toward reading a different, but plausible-looking, word.
- The central 'semantic gap' of handwriting synthesis: realistic strokes are far easier to generate than guaranteed-correct spelling. CTC guidance reduces but doesn't close this gap at the available data scale.

VISUAL & ARCHITECTURE BLUEPRINT

Visual-vs-Semantic Gap Diagram

Two-axis illustration: X-axis 'Visual Realism (high)', Y-axis 'Semantic Correctness (low)' — plot the GAN's output high on realism, low on correctness, with a sample generated word next to a callout comparing 'intended word' vs. 'what TrOCR actually reads back'.

5.3.3 Analysis — Why the LDM ControlNet Pipeline Failed

SLIDE TEXT CONTENT

- Produced distorted, blurry text failing OCR read tests, for 4 compounding reasons.
- (1) Stochastic stroke sensitivity — iterative denoising introduces high-frequency spatial shifts; a few-pixel deviation can break stroke connectivity or merge letters, fatal for handwriting.
- (2) CLIP vocabulary misalignment — trained on English web captions, zero structural representation of Devanagari, so even transliterated prompts can't map to correct glyph layouts.
- (3) Data-scale constraints — sharp ControlNet stroke edges need hundreds of thousands of clean pairs, far more than available Indic data. (4) Compute/gradient overhead — fine-tuning the SD U-Net under MPS suffered high gradient instability vs. direct GAN training. Conclusion: the GAN was more practical for this task and hardware.

VISUAL & ARCHITECTURE BLUEPRINT

Four-Cause Failure Breakdown

2x2 grid of failure-cause cards with icons: jittery-line (stroke sensitivity), CLIP logo crossed with a Devanagari letter + red X (vocabulary misalignment), dataset-size bar comparison (data scale), slow-spinning-gradient icon (compute overhead). Center: a sample blurry/distorted generated word as 'exhibit A'.

SLIDE 60

6.1 Conclusion

SLIDE TEXT CONTENT

- DevGen successfully integrates a hybrid, smart-routed OCR reader with a conditional word generator, delivered as a complete full-stack application.
- The flood-fill router optimizes hardware use, sending isolated symbols/digits to a lightweight CNN while reserving TrOCR for words. LoRA cut trainable parameters to 2.45%, making local MPS training feasible, reaching best validation CER 0.1380.
- Key lesson: recognition quality depends strongly on matching inference preprocessing to the training word-crop distribution — foreground cropping materially improved real-upload results.
- StyleGAN-Attention with CTC feedback stabilized and clearly outperformed LDM ControlNet, though a semantic gap remains; the CNN's collapse on segmented crops confirms segmentation-free recognition is the right architectural choice.

VISUAL & ARCHITECTURE BLUEPRINT

Achievement Summary Scorecard

Trophy-style scorecard with 4 headline stat tiles: 'CER 0.1380', '2.45% Params Trained', '98.2% Clean CNN Accuracy', 'GAN > LDM (stability winner)', each with a supporting icon, arranged around a central 'DevGen — Full-Stack Hybrid Neuro-Generative System' banner.

6.2 Future Recommendations

SLIDE TEXT CONTENT

- Improve ControlNet by replacing CLIP with a Devanagari-native text encoder (e.g. IndicBERT) and binary distance-field conditioning for cleaner, connected lines.
- Add layout parsing — text-line and word detection for multi-word/full-page reading. Strengthen augmentation with elastic deformation, perspective transforms, blur, brightness/contrast jitter, then use the stabilized GAN to synthesize more labeled words.
- Run full evaluation — complete validation/test-set CER for the best checkpoint (not a 1,000-sample subset), compared against base checkpoint and full fine-tuning.
- Close the semantic gap (heavier CTC weighting / recognizer-in-the-loop reward); calibrate confidence against validation reliability; replace rule-based NER with a trained Devanagari NER model.

VISUAL & ARCHITECTURE BLUEPRINT

Future Roadmap Graphic

Forward-pointing arrow/roadmap with 7 milestone flags labeled per recommendation, color-coded by theme (generation improvements, evaluation rigor, NER upgrade) signaling 'next semester's work'.

SLIDE 62

References & Key Resources

SLIDE TEXT CONTENT

- Core methods: TrOCR (Li et al. 2023), LoRA (Hu et al. 2022), StyleGAN2 (Karras et al. 2020), SAGAN (Zhang et al. 2019), Latent Diffusion (Rombach et al. 2022), ControlNet (Zhang et al. 2023), CLIP (Radford et al. 2021), CTC (Graves et al. 2006), Spectral Norm (Miyato et al. 2018), DiffAugment (Zhao et al. 2020), TTUR/FID (Heusel et al. 2017), ViT (Dosovitskiy et al. 2021), Transformer (Vaswani et al. 2017), DHCD (Acharya et al. 2015), Nepali GAN handwriting (Chhatkuli et al. 2021).
- Key datasets/models: c3rl/IIIT-INDIC-HW-WORDS-Hindi, paudelani/trocr-devanagari-2, DHCD (UCI repository).
- Key frameworks: PyTorch (MPS backend), Hugging Face PEFT/Diffusers/Transformers, FastAPI, React+Vite.

VISUAL & ARCHITECTURE BLUEPRINT

Citation Wall / Resource Grid

Tiled grid of small paper-card thumbnails, each with the paper's short name and year, grouped into 3 labeled zones: 'Recognition Methods', 'Generative Methods', 'Datasets & Frameworks' — a references poster wall for the deck's close.

Source: DevGen_Report.docx · Generated as an instructional-design pass over the full report text, front matter through appendices.